Introduction to Nextflow

Jakub Koubele

March 11, 2026

Nextflow is...

nextflow

Nextflow is...

- **workflow management system** used to organize and execute bioinformatics pipelines.

Nextflow is...

- **workflow management system** used to organize and execute bioinformatics pipelines.
- **domain-specific language** (DSL) built on Groovy and running on the Java Virtual Machine (JVM).

Nextflow is...

nextflow

- **workflow management system** used to organize and execute bioinformatics pipelines.
- **domain-specific language** (DSL) built on Groovy and running on the Java Virtual Machine (JVM).
- **powering nf-core**, a collection of high-quality, standardized bioinformatics pipelines.

nf-core

Why workflow management?

Many bioinformatics pipelines face these challenges:

Why workflow management?

Many bioinformatics pipelines face these challenges:

- Using many **heterogeneous tools** (command-line programs, R/Python scripts, etc.) that must be "glued" together.

Why workflow management?

Many bioinformatics pipelines face these challenges:

- Using many **heterogeneous tools** (command-line programs, R/Python scripts, etc.) that must be "glued" together.
  - Different tools have different requirements — packages and environments need to be managed.

Why workflow management?

Many bioinformatics pipelines face these challenges:

- Using many **heterogeneous tools** (command-line programs, R/Python scripts, etc.) that must be "glued" together.
  - Different tools have different requirements — packages and environments need to be managed.
- **Data are big** → parallelization and/or execution on HPC is desired.

Why workflow management?

Many bioinformatics pipelines face these challenges:

- Using many **heterogeneous tools** (command-line programs, R/Python scripts, etc.) that must be "glued" together.
  - Different tools have different requirements — packages and environments need to be managed.
- **Data are big** → parallelization and/or execution on HPC is desired.
- Pipelines form **dependency graphs** (not just step 1 → step 2 → step 3 etc.).

Why workflow management?

Many bioinformatics pipelines face these challenges:

- Using many **heterogeneous tools** (command-line programs, R/Python scripts, etc.) that must be "glued" together.
  - Different tools have different requirements — packages and environments need to be managed.
- **Data are big** → parallelization and/or execution on HPC is desired.
- Pipelines form **dependency graphs** (not just step 1 → step 2 → step 3 etc.).
  - The execution order should be resolved automatically (tasks may finish asynchronously).

Why workflow management?

Many bioinformatics pipelines face these challenges:

- Using many **heterogeneous tools** (command-line programs, R/Python scripts, etc.) that must be "glued" together.
  - Different tools have different requirements — packages and environments need to be managed.
- **Data are big** → parallelization and/or execution on HPC is desired.
- Pipelines form **dependency graphs** (not just step 1 → step 2 → step 3 etc.).
  - The execution order should be resolved automatically (tasks may finish asynchronously).
  - If part of the pipeline changes, re-run only affected downstream steps (caching).

How Nextflow helps

How Nextflow helps

- Language for "gluing" tools into pipelines. Basic components: **processes** and **channels**.
  - **Processes**: define computational tasks (scripts, command-line tools, etc.).
  - **Channels**: transport data between processes (in asynchronous queues).

How Nextflow helps

- Language for "gluing" tools into pipelines. Basic components: **processes** and **channels**.
  - **Processes**: define computational tasks (scripts, command-line tools, etc.).
  - **Channels**: transport data between processes (in asynchronous queues).
- Supports software environments: processes can run in containers (Docker, Singularity, etc.) or environments (Conda, modules, etc.).

How Nextflow helps

- Language for "gluing" tools into pipelines. Basic components: **processes** and **channels**.
  - **Processes**: define computational tasks (scripts, command-line tools, etc.).
  - **Channels**: transport data between processes (in asynchronous queues).
- Supports software environments: processes can run in containers (Docker, Singularity, etc.) or environments (Conda, modules, etc.).
- Channels are asynchronous $\rightarrow$ automatic parallelization and flexible execution order.

How Nextflow helps

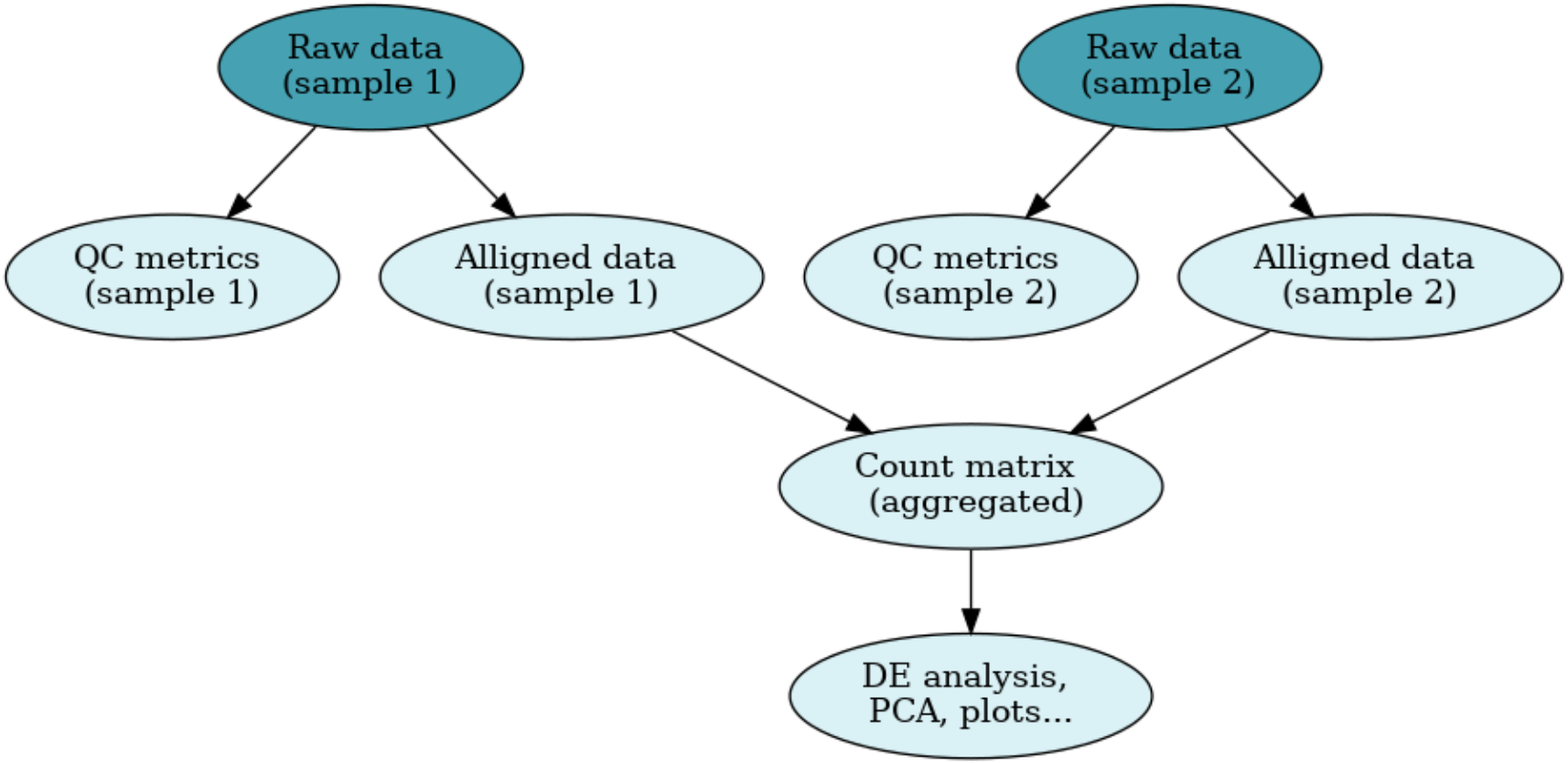
- Language for "gluing" tools into pipelines. Basic components: **processes** and **channels**.
  - **Processes**: define computational tasks (scripts, command-line tools, etc.).
  - **Channels**: transport data between processes (in asynchronous queues).
- Supports software environments: processes can run in containers (Docker, Singularity, etc.) or environments (Conda, modules, etc.).
- Channels are asynchronous $\rightarrow$ automatic parallelization and flexible execution order.
- Pipeline logic is separated from execution $\rightarrow$ easy deployment on HPC or cloud infrastructure.

Pipelines as DAGs

- Computational pipelines can be represented as **directed acyclic graphs (DAGs)**.

Pipelines as DAGs

- Computational pipelines can be represented as **directed acyclic graphs (DAGs)**.
  - **Nodes**: represent data or intermediate results.
  - **Edges**: represent dependencies between nodes; a child node is computed from the data in its parent nodes.

Pipelines as DAGs

- Computational pipelines can be represented as **directed acyclic graphs (DAGs)**.
  - **Nodes**: represent data or intermediate results.
  - **Edges**: represent dependencies between nodes; a child node is computed from the data in its parent nodes.